Taskanium — Complete Locked Workflow

> Single source of truth. Build from this. Do not change without updating here. AI Hackfest 2026 · Deadline: Apr 19 @ 7:30pm IST · 4-person team

---

✅ Locked Decisions

Decision	Choice
App name	Taskanium
Platform	Electron (always-on-top desktop widget)
Frontend	React + Vite + Tailwind + Framer Motion
Backend	FastAPI (Python)
Session input	Task text + Energy level (Low / Med / High)
Reminder system	3 phases + doom spiral escalation + hyperfocus detection
Voice strategy	Gemini generates fresh message every reminder via ElevenLabs
Session end	Done ✓ / Abandon ✗ buttons
Sponsors	Gemini + ElevenLabs + Snowflake + DigitalOcean
Insights	Electron → shell.openExternal → DO-deployed web page
Live URL	FastAPI + Insights page deployed on DigitalOcean App Platform

---

🧠 The Core Insight

> ADHD is not a "forgetting to do things" problem. It is an **interest-based nervous system** problem. No existing productivity app understands this. Taskanium does.

5 Judge-Visible Differentiators

1. **Phase-based system** — 3 phases (Start → Mid → End), guidance changes at each stage
2. **Body doubling** — ElevenLabs voice creates the psychological effect of someone working alongside you
3. **Adaptive learning** — Snowflake past sessions fed into Gemini context; gets smarter each session

4. **Dynamic voice messages** — Gemini writes a fresh message every single reminder, never repetitive
5. **One thing at a time** — never shows a task list, only one next step; zero overwhelm

---

📁 Project Structure

```
taskanium/
├── electron/
│   ├── main.js              # Electron entry — window, IPC, alwaysOnTop
│   ├── preload.js           # IPC bridge: minimize, expand, open-insights
│   └── package.json         # Electron + electron-builder config
│
├── frontend/                # React app — runs inside Electron renderer
│   ├── src/
│   │   ├── App.jsx          # Root: TaskInput | ActiveSession | SessionEnd
│   │   ├── components/
│   │   │   ├── TaskInput.jsx          # Screen 1: task + energy entry
│   │   │   ├── FloatingBubble.jsx     # Screen 2: widget (panel + bubble modes)
│   │   │   ├── MicroStep.jsx          # Current step + phase indicator
│   │   │   ├── ReminderToast.jsx      # Expands on reminder, plays audio
│   │   │   ├── HyperfocusOverlay.jsx  # Silent mode indicator
│   │   │   └── SessionEnd.jsx         # Done / Abandon + save to Snowflake
│   │   ├── hooks/
│   │   │   ├── useAdaptiveTimer.js    # 3-phase + doom spiral + hyperfocus logic
│   │   │   └── useAudio.js            # ElevenLabs base64 audio playback
│   │   ├── api.js           # All fetch() calls to FastAPI backend
│   │   └── main.jsx
│   ├── index.html
│   ├── vite.config.js
│   └── tailwind.config.js
│
├── backend/
│   ├── main.py              # FastAPI app, all routes + CORS
│   ├── gemini_client.py     # All Gemini prompts + API calls
│   ├── elevenlabs_client.py # TTS → base64 audio
│   ├── snowflake_client.py  # Session CRUD + insights queries
│   ├── models.py            # Pydantic request/response models
│   ├── requirements.txt
│   └── .env.example
│
├── insights/                # Standalone React page — deployed on DigitalOcean
│   ├── src/
│   │   └── App.jsx          # GET /insights → recharts dashboard
│   ├── index.html
```

```
│   └── vite.config.js
│
├── .do/
│   └── app.yaml              # DigitalOcean: backend + insights deployment
└── README.md
```

🔁 Full System Flow

```
User opens Taskanium widget (Electron, always-on-top)
        ↓
SCREEN 1: Enter task + energy level
        ↓
POST /start-session
  → Fetch last 5 sessions from Snowflake
  → Build context summary
  → Send to Gemini with task + energy + history
  → Gemini returns: first_step, estimated_minutes,
    phase_boundaries, base_interval, opening_voice_text
        ↓
SCREEN 2: Widget shows first step + opening voice plays
  → Widget auto-minimises to floating bubble (64×64px)
  → useAdaptiveTimer starts
        ↓
    ┌──────────────────────────────────────┐
    │          ADAPTIVE TIMER LOOP         │
    │                                      │
    │  Every tick: evaluate state          │
    │                                      │
    │  → NORMAL     → fire reminder at     │
    │                 base × phase_mult    │
    │                                      │
    │  → DOOM SPIRAL → escalate interval,  │
    │                  urgent-gentle tone  │
    │                                      │
    │  → HYPERFOCUS  → go silent,          │
    │                  shrink bubble       │
    └──────────────────────────────────────┘
        ↓
  [REMINDER FIRES]
  POST /speak
    → Gemini generates fresh phase-aware message
    → ElevenLabs TTS → base64 audio
  Bubble glows → expands → voice plays → auto-shrinks
        ↓
```

```
User continues working
        ↓
[SESSION ENDS — Done ✓ or Abandon ✗]
POST /end-session → Snowflake saves all data
Closing message shown → returns to Screen 1
        ↓
Next session: past data improves Gemini's decisions
```

📋 Feature-by-Feature Spec

FEATURE 1 — Task Input + Gemini Micro-Step Engine

Screen: TaskInput.jsx **User enters:** Task text + energy level (Low / Medium / High)

POST /start-session request:

```json
{
  "task_text": "Write essay intro",
  "energy_level": "low"
}
```

Backend steps:

1. Fetch last 5 sessions from Snowflake
2. Build context: "User completed 3/5 recent sessions. Abandoned 2 at start phase."
3. Send to Gemini with task + energy + context

Gemini system prompt:

```
You are Taskanium, an AI assistant designed specifically for ADHD brains.

Task: {task_text}
Energy level: {energy_level} (low / medium / high)
Past sessions: {last_5_sessions_summary}

Return ONLY valid JSON, no markdown:
{
  "first_step": "One action under 12 words. Low energy = tiny (just open the
file). High energy = real chunk.",
  "estimated_minutes": <realistic integer>,
  "phase_boundaries": [<min when phase 1 ends>, <min when phase 2 ends>],
  "base_interval_minutes": <5 for low, 15 for medium, 25 for high>,
  "opening_voice_text": "Warm 1-sentence opening. Acknowledge low energy if
applicable."
}

Rules:
- Never use productivity jargon
- Tone: warm friend, not life coach
- Low energy = acknowledge that starting IS the win
- Adjust estimates based on past session history if provided
```

POST /start-session response:

```json
{
  "session_id": "uuid-here",
  "first_step": "Just open a blank doc and write one sentence.",
  "estimated_minutes": 25,
  "phase_boundaries": [8, 18],
  "base_interval_minutes": 5,
  "opening_voice_text": "Let's just start small… you've got this."
}
```

FEATURE 2 — ElevenLabs Voice Body Doubling

On every reminder — POST /speak request:

```json
```

```
{
  "session_id": "uuid",
  "task_text": "Write essay intro",
  "energy_level": "low",
  "current_phase": 0,
  "reminder_number": 2,
  "is_doom_spiral": false
}
```

Gemini generates a fresh spoken message based on phase:

Phase	Tone	Example
0 — Start	Warm, tiny step	"Let's just open the file… that's all you need right now."
1 — Mid	Brief check-in	"Still with you. How's it going? One small step is enough."
2 — End	Minimal	"Almost there. You're in the home stretch."
Doom spiral	Gentle urgency	"Hey — still there? Even the tiniest move forward counts."

Key rules:

- Message freshly generated every time — never hardcoded, never repeated
- Gemini varies wording naturally each call
- Never judgmental, never pushy

ElevenLabs config:

```
Voice ID:          21m00Tcm4TlvDq8ikWAM  (Rachel — calm, warm)
Model:             eleven_monolingual_v1
Stability:         0.75
Similarity boost:  0.85
```

POST /speak response:

```json
{
  "audio_base64": "base64-encoded-mp3...",
  "message_text": "Let's just open the file… that's all you need right now."
}
```

Frontend plays via:

```javascript
const audio = new Audio(`data:audio/mp3;base64,${audio_base64}`)
audio.play()
```

FEATURE 3 — Smart Adaptive Reminder System

Hook: useAdaptiveTimer.js

Timing formula:

```
base_interval    = { low: 5, medium: 15, high: 25 }[energy_level]  // minutes
phase_multiplier = [1.0, 0.65, 0.35][current_phase]
current_interval = base_interval × phase_multiplier
```

Phase transitions (from Gemini's phase_boundaries):

```
Phase 0 (Start):  0 min → phase_boundaries[0]       e.g. 0 → 8 min
Phase 1 (Mid):    phase_boundaries[0] → [1]          e.g. 8 → 18 min
Phase 2 (End):    phase_boundaries[1] → estimated    e.g. 18 → 25 min
```

Low energy example (base=5min, task=25min):

```
Phase 0:  5 × 1.00 = 5.0 min   ← frequent help while starting
Phase 1:  5 × 0.65 = 3.25 min
Phase 2:  5 × 0.35 = 1.75 min  ← barely interrupts at end
```

High energy example (base=25min, task=25min):

```
Phase 0:  25 × 1.00 = 25 min   ← barely interrupts
Phase 1:  25 × 0.65 = 16 min
Phase 2:  25 × 0.35 = 8.75 min
```

Hook returns:

```javascript
```

```
{
  currentPhase: 0 | 1 | 2,
  secondsUntilReminder: number,
  missedReminders: number,
  isDoomSpiral: boolean,
  isHyperfocus: boolean,
  elapsedMinutes: number
}
```

FEATURE 4 — Doom Spiral Detection

Trigger:

```
missedReminders >= 2
(user has not tapped/dismissed last 2 reminders)
```

What happens:

1. `isDoomSpiral = true`
2. Interval multiplied by 0.5 (reminders come faster)
3. POST /speak sends `is_doom_spiral: true`
4. Gemini writes gentler, more present message
5. After 3 consecutive missed reminders → voice suggests a break: *"It's okay to take a 5-minute break. That's not giving up."*

Reset: User taps/dismisses any reminder → `missedReminders = 0`

FEATURE 5 — Hyperfocus Mode

Purpose: Detects when user is in deep flow and silences completely. This is Taskanium's most ADHD-native feature.

Detection (all conditions must be true):

```
elapsed_time > phase_boundaries[0]      (past start phase — session is real)
missedReminders >= 3                    (ignoring reminders consistently)
time_since_last_interaction > base_interval × 2.5
isDoomSpiral == false                   (NOT stuck — actively working)
```

Key distinction:

```
Doom spiral  = user is STUCK  → escalate, be present
Hyperfocus   = user is FLOWING → go silent, step back
```

What happens:

1. `isHyperfocus = true`
2. All regular reminders stop
3. Voice goes completely silent
4. Window shrinks: `win.setSize(40, 40)` — tiny dot in corner
5. HyperfocusOverlay shows: faint 🔵 indicator
6. Only one check-in fires after `base_interval × 3`:
   - Text only (no voice): *"Still on track?"*

Exit: User taps bubble → hyperfocus exits, normal session resumes

---

FEATURE 6 — Floating Bubble Widget UI

Electron window config:

```javascript
{
  width: 320, height: 420,
  x: screenWidth - 340,
  y: screenHeight - 440,
  frame: false,
  transparent: true,
  alwaysOnTop: true,
  skipTaskbar: true,
  resizable: false
}
win.setAlwaysOnTop(true, 'floating')  // stays above fullscreen apps
win.setVisibleOnAllWorkspaces(true)
```

5 UI States:

```
STATE 1: IDLE
  320×420px · TaskInput form · indigo palette

STATE 2: ACTIVE EXPANDED
  320×420px · Task + MicroStep + phase [●●○] + countdown + minimize btn

STATE 3: ACTIVE BUBBLE  ← default working state
  64×64px · bottom-right corner · soft pulse animation
  scale 1→1.06→1, opacity 0.8→1→0.8, 4s loop

STATE 4: REMINDER FIRING
  Bubble glows (indigo ring, 0.8s)
  Expands to show ReminderToast + message text
  Audio plays
  Auto-returns to STATE 3 after 8s or tap

STATE 5: HYPERFOCUS
  40×40px · extremely faint pulse · 🔵 only
  No voice · No reminders · Single text check-in

STATE 6: SESSION END
  320×420px · Done ✓ / Abandon ✗ · closing message
  Returns to STATE 1 after 3s
```

IPC handlers:

```javascript
ipcMain.on('minimize-to-bubble', () => {
  win.setSize(64, 64)
  win.setPosition(screenWidth - 84, screenHeight - 84)
})
ipcMain.on('minimize-to-hyperfocus', () => {
  win.setSize(40, 40)
  win.setPosition(screenWidth - 56, screenHeight - 56)
})
ipcMain.on('expand-to-panel', () => {
  win.setSize(320, 420)
  win.setPosition(screenWidth - 340, screenHeight - 440)
})
ipcMain.on('open-insights', () => {
  shell.openExternal('https://taskanium.ondigitalocean.app/insights')
})
```

FEATURE 7 — Snowflake Session Memory

Table schema:

```sql
CREATE TABLE sessions (
  id                      VARCHAR(36) DEFAULT UUID_STRING() PRIMARY KEY,
  task_text               VARCHAR(500),
  energy_level            VARCHAR(10),
  estimated_mins          INTEGER,
  actual_mins             INTEGER,
  reminders_sent          INTEGER,
  reminders_acknowledged  INTEGER,
  completed               BOOLEAN,
  abandoned_at_phase      VARCHAR(10),   -- 'start', 'mid', 'end', null
  hyperfocus_detected     BOOLEAN,
  started_at              TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  ended_at                TIMESTAMP_NTZ
);
```

Gemini context injection (every /start-session):

```
"Past session context:
- User completed 3 of last 5 sessions
- Avg actual time for medium-energy: 22 min (estimated 18)
- Abandoned 2 sessions at start phase (low energy)
- Hyperfocus detected in 2 sessions
Adjust first_step size and base_interval accordingly."
```

FEATURE 8 — Insights Dashboard

Deployed: DigitalOcean App Platform **URL:** `taskanium.ondigitalocean.app/insights` ← judge-accessible live URL **Opened from Electron:** `shell.openExternal(url)`

GET /insights returns:

```json
```

```
{
  "total_sessions": 12,
  "completion_rate": 0.75,
  "avg_duration_by_energy": { "low": 14, "medium": 22, "high": 32 },
  "sessions_by_day": [{ "date": "2026-04-18", "count": 3 }],
  "abandoned_phases": { "start": 2, "mid": 1, "end": 0 },
  "hyperfocus_sessions": 3,
  "avg_reminders_per_session": 4.2
}
```

Dashboard charts (recharts):

- Completion rate % — hero metric (big number)
- Sessions per day — AreaChart
- Avg actual vs estimated by energy — BarChart
- Abandonment by phase — PieChart
- Hyperfocus sessions count

🔌 All API Endpoints

Endpoint	Method	In	Out	Does
/start-session	POST	task_text, energy_level	session_id, first_step, estimated_minutes, phase_boundaries, base_interval_minutes, opening_voice_text	Snowflake fetch → Gemini → Snowflake create
/speak	POST	session_id, task_text, energy_level, current_phase, reminder_number, is_doom_spiral	audio_base64, message_text	Gemini message → ElevenLabs TTS → audio
/end-session	POST	session_id, completed, actual_minutes, reminders_sent, reminders_acknowledged, abandoned_at_phase, hyperfocus_detected	{ ok: true }	Snowflake UPDATE
/insights	GET	—	stats object	Snowflake aggregation queries

Endpoint	Method	In	Out	Does
/health	GET	—	{ status: "ok" }	DO health check

🎨 Design Language

```
Primary:     #6366f1  (indigo — calm, not clinical)
Secondary:   #818cf8  (lighter indigo)
Background:  #fafaff  (near-white, very soft)
Dark bg:     #1e1e2e  (deep purple-black)
Text:        #1e1b4b  (dark indigo, not harsh black)
Success:     #10b981  (green — done)
Neutral:     #94a3b8  (grey — abandon)

Typography:  Inter, system-ui · line-height: 1.6
Radius:      16px widget · 12px buttons · 999px bubble

Rules:
  × No reds anywhere
  × No task lists during session
  × No aggressive CTAs
  ✓ One thing on screen at a time
  ✓ Always calm, never urgent-looking
```

👥 Team Split

Person	Owns	Key files
P1 — Backend	FastAPI endpoints, Gemini prompts, ElevenLabs client	main.py, gemini_client.py, elevenlabs_client.py, models.py
P2 — Data	Snowflake schema + queries, insights dashboard UI	snowflake_client.py, insights/src/App.jsx
P3 — Frontend	Electron main/preload, all React components, useAdaptiveTimer	electron/, frontend/src/
P4 — Ship	api.js wiring, DO deploy, electron-builder, demo video, Devpost	api.js, .do/app.yaml, README, all submission materials

⏱️ 24-Hour Build Order

```
Hr 0-1    ALL: Setup
          GitHub repo · API keys (Gemini, ElevenLabs, Snowflake, DO)
          Electron scaffold · Vite scaffold · FastAPI scaffold

Hr 1-3    P1: POST /start-session (Gemini prompt + Snowflake fetch)
          P2: Snowflake schema + connector + last-5 query
          P3: electron/main.js (alwaysOnTop window + all IPC handlers)

Hr 3-5    P1: POST /speak (Gemini phase message + ElevenLabs TTS)
          P2: GET /insights + aggregation queries
          P3: TaskInput.jsx + FloatingBubble.jsx (full panel + bubble)

Hr 5-7    P1: POST /end-session + GET /health → push to DO
          P2: Insights dashboard UI with recharts
          P3: useAdaptiveTimer hook (3-phase + doom spiral + hyperfocus)
          P4: api.js — all fetch wrappers

Hr 7-9    P3: ReminderToast + useAudio + HyperfocusOverlay + SessionEnd
          P4: Wire all components to api.js — full flow working locally
          P2: Deploy insights page to DO App Platform

Hr 9-11   ALL: Integration testing — full flow 5× end-to-end
          Fix: audio issues, timer bugs, Snowflake timeouts, IPC issues

Hr 11-14  P3: Framer Motion polish (bubble animations, glow, transitions)
          P4: electron-builder → Taskanium-Setup.exe → GitHub Release

Hr 14-16  P4: Record 2-min demo video
          P4: Write Devpost submission

Hr 16-18  P1: Write README.md
          ALL: Final checklist → SUBMIT before 7:30pm IST
```

🎬 Demo Video Script (2 minutes)

0:00–0:08 — Intro

> "Hey, I'm [name] and this is my demo for AI Hackfest. This is Taskanium — an AI companion built specifically for ADHD brains."

0:08–0:25 — Core loop

0:25–0:50 — Voice + body doubling

> Widget minimises to bubble. Trigger reminder. Bubble glows. ElevenLabs plays.
> "That's ElevenLabs — a calm voice working beside you. This is body doubling — proven effective for ADHD."

0:50–1:10 — Phase system

> "The system divides your session into 3 phases. It gives more help at the start and backs off as you get into flow. It learns when to help and when to stay silent."

1:10–1:25 — Hyperfocus mode

> Show hyperfocus triggering — widget shrinks to tiny dot.
> "If Taskanium detects deep focus, it goes completely silent. No interruptions. Most apps break your flow. Taskanium chooses silence."

1:25–1:40 — Snowflake insights

> Click insights → browser opens DO page. Show session chart.
> "Every session feeds back into Gemini's context. It gets smarter every time."

1:40–1:55 — Closing

> Done button. Closing message.
> "366 million adults have ADHD. No app understands it's an interest-based nervous system problem — not a forgetting problem. Taskanium does."

1:55–2:00

> "Built in 24 hours by [team] for AI Hackfest. Thank you."

✅ Final Submission Checklist

Code

- [ ] GitHub repo PUBLIC
- [ ] API keys in env vars only — never committed
- [ ] .env.example present
- [ ] AI tools disclosed in Devpost

Deployment

- [ ] FastAPI live on DigitalOcean
- [ ] Insights page live at taskanium.ondigitalocean.app/insights
- [ ] GET /health returns 200

- [ ] shell.openExternal opens insights in browser

Widget

- [ ] alwaysOnTop confirmed — stays above VS Code
- [ ] ElevenLabs voice plays on reminder
- [ ] Hyperfocus mode triggers and shrinks widget
- [ ] Done ✓ / Abandon ✗ both save to Snowflake
- [ ] GitHub Release has Taskanium-Setup.exe

Video

- [ ] Starts: "Hey I'm [name] and this is my demo for AI Hackfest"
- [ ] Under 2 minutes
- [ ] Audio clearly audible
- [ ] Shows widget above VS Code
- [ ] Shows hyperfocus mode
- [ ] Shows Snowflake insights page
- [ ] Created this weekend

Devpost

- [ ] All 4 sponsors named with exact usage explained
- [ ] 5 differentiators mentioned
- [ ] Live insights URL + GitHub Release link in README

🔑 Environment Variables

```bash
# backend/.env
GEMINI_API_KEY=
ELEVENLABS_API_KEY=
ELEVENLABS_VOICE_ID=21m00Tcm4TlvDq8ikWAM

SNOWFLAKE_ACCOUNT=
SNOWFLAKE_USER=
SNOWFLAKE_PASSWORD=
SNOWFLAKE_DATABASE=TASKANIUM_DB
SNOWFLAKE_SCHEMA=PUBLIC
SNOWFLAKE_WAREHOUSE=COMPUTE_WH

# frontend/.env
VITE_API_URL=https://your-backend.ondigitalocean.app
```

🏆 Prize Targeting Summary

Prize	How Taskanium wins it
Best Use of Gemini	Gemini makes ALL decisions: first step, phase messages, doom spiral, hyperfocus inference, learning from past sessions. Fresh message generated every reminder.
Best Use of ElevenLabs	Voice is the emotional core. Body doubling through fresh, phase-aware messages. Hyperfocus silences it — making its presence feel more meaningful when it does speak.
Best Use of Snowflake	Every session stored. Last 5 sessions injected into Gemini context. Live insights dashboard queries Snowflake. Snowflake = why it gets smarter.
Best Use of DigitalOcean	FastAPI + insights page on DO App Platform. Push-to-deploy. The insights page is the judge-accessible live URL.

Version: LOCKED · AI Hackfest build weekend, April 2026 Team: Taskanium · 4 people This is the single source of truth. All code must match this spec.